

Trabalho Computacional 3: CIFAR-10 com PyTorch e Transfer Learning

Aluno: Luidgi Varela Carneiro - **Matrícula:** 231011669

Aluno: Marcos Paulo De Oliveira - **Matrícula:** 221020889

Notebook completo para classificação de imagens do CIFAR-10 usando PyTorch e PyTorch Lightning.

Trabalho Computacional 3. Rede Convolutacional e Transfer Learning

Notebook completo para classificação de imagens do CIFAR-10 usando PyTorch e PyTorch Lightning.

Experimentos implementados:

Modelo	Descrição
MLP	Perceptron multicamadas com duas camadas escondidas
VGG16	Transfer learning com VGG16 pré-treinada
ResNet18	Transfer learning com ResNet18 pré-treinada
ResNet18 + Dropout	ResNet18 com Dropout(0.5) entre camadas densas
ResNet18 + L2	ResNet18 com regularização L2 via <code>weight_decay</code>
ResNet18 + L1	ResNet18 com penalização L1 adicionada à função de perda

```
In [1]: %pip -q install pytorch-lightning torchmetrics
```

```
In [2]: import contextlib
import io
import os
import random

import numpy as np
import pandas as pd
import torch
import torch.nn as nn
import torchvision
import torchvision.transforms as transforms
import pytorch_lightning as pl

from IPython.display import Markdown, display
```



```

full_train = torchvision.datasets.CIFAR10(
    root=root, train=True, transform=trans, download=True
)

train_set_size = int(len(full_train) * 0.8)
valid_set_size = len(full_train) - train_set_size
generator = torch.Generator().manual_seed(SEED)

self.train, self.val = torch.utils.data.random_split(
    full_train,
    [train_set_size, valid_set_size],
    generator=generator,
)

self.test = torchvision.datasets.CIFAR10(
    root=root, train=False, transform=trans, download=True
)

with contextlib.redirect_stdout(io.StringIO()):
    dataset = CIFAR10(root=DATA_DIR)

train_dataloader = torch.utils.data.DataLoader(
    dataset.train,
    batch_size=BATCH_SIZE,
    shuffle=True,
    num_workers=2,
    pin_memory=PIN_MEMORY,
)
val_dataloader = torch.utils.data.DataLoader(
    dataset.val,
    batch_size=BATCH_SIZE,
    shuffle=False,
    num_workers=2,
    pin_memory=PIN_MEMORY,
)
test_dataloader = torch.utils.data.DataLoader(
    dataset.test,
    batch_size=BATCH_SIZE,
    shuffle=False,
    num_workers=2,
    pin_memory=PIN_MEMORY,
)

print(f"Treino: {len(dataset.train)} imagens")
print(f"Validação: {len(dataset.val)} imagens")
print(f"Teste: {len(dataset.test)} imagens")

```

Treino: 40000 imagens
Validação: 10000 imagens
Teste: 10000 imagens

Classe Lightning e funções auxiliares

A classe abaixo encapsula treino, validação, teste, otimização com Adam e, quando solicitado, regularização L1 e L2. Todos os experimentos usam GPU quando disponível, `EarlyStopping` e `max_epochs=10`.

```
In [4]: class LightModel(pl.LightningModule):
    def __init__(self, model, lr=1e-3, weight_decay=0.0, l1_lambda=0.0):
        super().__init__()
        self.model = model
        self.lr = lr
        self.weight_decay = weight_decay
        self.l1_lambda = l1_lambda

    def forward(self, x):
        return self.model(x)

    def training_step(self, batch, batch_idx):
        x, y = batch
        y_hat = self(x)
        loss = nn.functional.cross_entropy(y_hat, y)

        if self.l1_lambda > 0:
            l1_penalty = sum(
                param.abs().sum()
                for param in self.parameters()
                if param.requires_grad
            )
            loss = loss + self.l1_lambda * l1_penalty

        self.log("train_loss", loss, on_epoch=True, prog_bar=True)
        return loss

    def validation_step(self, batch, batch_idx):
        x, y = batch
        y_hat = self(x)
        loss = nn.functional.cross_entropy(y_hat, y)
        self.log("val_loss", loss, on_epoch=True, prog_bar=True)
        return loss

    def test_step(self, batch, batch_idx):
        x, y = batch
        y_hat = self(x)
        loss = nn.functional.cross_entropy(y_hat, y)
        preds = torch.argmax(y_hat, dim=1)
        acc = accuracy(preds, y, task="multiclass", num_classes=NUM_CLASSES)

        self.log("test_loss", loss, on_epoch=True, prog_bar=True)
        self.log("test_acc", acc, on_epoch=True, prog_bar=True)

    def configure_optimizers(self):
        trainable_params = [
            param for param in self.parameters() if param.requires_grad
        ]
        return torch.optim.Adam(
            trainable_params,
            lr=self.lr,
```

```

        weight_decay=self.weight_decay,
    )

def make_trainer():
    early_stopping = EarlyStopping(
        monitor="val_loss",
        patience=5,
        mode="min",
        min_delta=0.001,
    )

    return Trainer(
        accelerator=ACCELERATOR,
        devices=1,
        callbacks=[early_stopping],
        max_epochs=MAX_EPOCHS,
        log_every_n_steps=20,
        enable_checkpointing=False,
    )

def train_test_experiment(name, lightning_model, weight_file):
    print(f"\n===== {name} =====")
    trainer = make_trainer()
    trainer.fit(
        model=lightning_model,
        train_dataloaders=train_dataloader,
        val_dataloaders=val_dataloader,
    )

    weight_path = os.path.join(WEIGHTS_DIR, weight_file)
    torch.save(lightning_model.model.state_dict(), weight_path)
    lightning_model.model.load_state_dict(
        torch.load(weight_path, map_location="cpu")
    )

    test_metrics = trainer.test(
        model=lightning_model,
        dataloaders=test_dataloader,
        verbose=False,
    )[0]

    result = {
        "Modelo": name,
        "test_acc": float(test_metrics["test_acc"]),
        "test_loss": float(test_metrics["test_loss"]),
    }

    print("Resumo do teste:")
    print(f"  test_acc: {result['test_acc']:.4f}")
    print(f"  test_loss: {result['test_loss']:.4f}")
    print(f"  pesos:    {weight_path}")

    torch.cuda.empty_cache()
    return result

```

```
experiment_results = []
```

2. Treinando um MLP

O MLP recebe a imagem achatada por `Flatten` e usa duas camadas escondidas. Como CIFAR-10 é um problema visual com estrutura espacial importante, espera-se desempenho inferior ao das redes convolucionais.

```
In [5]: mlp_arch = nn.Sequential(
        nn.Flatten(),
        nn.Linear(3 * 224 * 224, 512),
        nn.ReLU(),
        nn.Linear(512, 128),
        nn.ReLU(),
        nn.Linear(128, NUM_CLASSES),
    )

mlp = LightningModel(mlp_arch, lr=1e-4)

experiment_results.append(
    train_test_experiment(
        name="MLP",
        lightning_model=mlp,
        weight_file="mlp_cifar10.pt",
    )
)
```

==== MLP =====

Resumo do teste:

```
test_acc: 0.4784
test_loss: 1.6286
pesos:     pesos/mlp_cifar10.pt
```

3. Uso da rede VGG16 pré-treinada

A VGG16 é carregada com pesos pré-treinados no ImageNet. Todas as camadas convolucionais ficam congeladas e somente o bloco classificador novo é treinado para as 10 classes do CIFAR-10.

Classificador solicitado:

```
Flatten
Linear(25088, 50)
ReLU
Linear(50, 20)
ReLU
Linear(20, 10)
```

```
In [6]: def build_vgg16_transfer():
        model = vgg16(weights=VGG16_Weights.DEFAULT, progress=True)

        for param in model.parameters():
            param.requires_grad = False

        model.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(25088, 50),
            nn.ReLU(),
            nn.Linear(50, 20),
            nn.ReLU(),
            nn.Linear(20, NUM_CLASSES),
        )

        return model

vgg16_transfer = LightningModel(build_vgg16_transfer(), lr=1e-3)

experiment_results.append(
    train_test_experiment(
        name="VGG16",
        lightning_model=vgg16_transfer,
        weight_file="vgg16_cifar10.pt",
    )
)
```

===== VGG16 =====

Resumo do teste:

test_acc: 0.8460

test_loss: 0.8671

pesos: pesos/vgg16_cifar10.pt

4. Extras

4.1 Outra rede convolucional

Nesta etapa é usada a ResNet18 pré-treinada. As camadas convolucionais são congeladas e a camada final é substituída pelo classificador solicitado:

Linear(in_features, 50)

ReLU

Linear(50, 20)

ReLU

Linear(20, 10)

```
In [7]: def build_resnet18_transfer(dropout=False):
        model = resnet18(weights=ResNet18_Weights.DEFAULT, progress=True)

        for param in model.parameters():
            param.requires_grad = False
```

```

in_features = model.fc.in_features

if dropout:
    model.fc = nn.Sequential(
        nn.Linear(in_features, 50),
        nn.ReLU(),
        nn.Dropout(0.5),
        nn.Linear(50, 20),
        nn.ReLU(),
        nn.Dropout(0.5),
        nn.Linear(20, NUM_CLASSES),
    )
else:
    model.fc = nn.Sequential(
        nn.Linear(in_features, 50),
        nn.ReLU(),
        nn.Linear(50, 20),
        nn.ReLU(),
        nn.Linear(20, NUM_CLASSES),
    )

return model

resnet18_transfer = LightModel(build_resnet18_transfer(), lr=1e-3)

experiment_results.append(
    train_test_experiment(
        name="ResNet18",
        lightning_model=resnet18_transfer,
        weight_file="resnet18_cifar10.pt",
    )
)

```

===== ResNet18 =====

Resumo do teste:

test_acc: 0.8109

test_loss: 0.5583

pesos: pesos/resnet18_cifar10.pt

4.2 Regularização

Os modelos abaixo são treinados e testados separadamente, sempre partindo de uma nova ResNet18 pré-treinada com camadas convolucionais congeladas.

4.2.1 Dropout

Adiciona `Dropout(0.5)` entre as camadas densas da ResNet18.

```

In [8]: resnet18_dropout = LightModel(build_resnet18_transfer(dropout=True), lr=1e-3)

experiment_results.append(
    train_test_experiment(
        name="ResNet18 + Dropout",
        lightning_model=resnet18_dropout,
    )
)

```

```

        weight_file="resnet18_dropout_cifar10.pt",
    )
)

```

==== ResNet18 + Dropout =====

Resumo do teste:

```

test_acc: 0.7077
test_loss: 0.9613
pesos:     pesos/resnet18_dropout_cifar10.pt

```

4.2.2 L2

A regularização L2 é implementada com o parâmetro `weight_decay` do otimizador Adam.

```

In [9]: resnet18_l2 = LightModel(
        build_resnet18_transfer(),
        lr=1e-3,
        weight_decay=1e-4,
    )

    experiment_results.append(
        train_test_experiment(
            name="ResNet18 + L2",
            lightning_model=resnet18_l2,
            weight_file="resnet18_l2_cifar10.pt",
        )
    )

```

==== ResNet18 + L2 =====

Resumo do teste:

```

test_acc: 0.7986
test_loss: 0.5924
pesos:     pesos/resnet18_l2_cifar10.pt

```

4.2.3 L1

A regularização L1 é adicionada manualmente à função de perda durante o

`training_step`.

```

In [10]: resnet18_l1 = LightModel(
        build_resnet18_transfer(),
        lr=1e-3,
        l1_lambda=1e-5,
    )

    experiment_results.append(
        train_test_experiment(
            name="ResNet18 + L1",
            lightning_model=resnet18_l1,
            weight_file="resnet18_l1_cifar10.pt",
        )
    )

```

```
===== ResNet18 + L1 =====
Resumo do teste:
  test_acc: 0.8026
  test_loss: 0.5772
  pesos:    pesos/resnet18_l1_cifar10.pt
```

Tabela comparativa

A tabela abaixo reúne `test_acc` e `test_loss` de todos os experimentos solicitados.

```
In [11]: results_df = pd.DataFrame(experiment_results)
results_df = results_df[["Modelo", "test_acc", "test_loss"]]
results_df["test_acc"] = results_df["test_acc"].round(4)
results_df["test_loss"] = results_df["test_loss"].round(4)
display(results_df)
```

	Modelo	test_acc	test_loss
0	MLP	0.4784	1.6286
1	VGG16	0.8460	0.8671
2	ResNet18	0.8109	0.5583
3	ResNet18 + Dropout	0.7077	0.9613
4	ResNet18 + L2	0.7986	0.5924
5	ResNet18 + L1	0.8026	0.5772

Conclusão

```
In [12]: best_acc_row = results_df.loc[results_df["test_acc"].idxmax()]
lowest_loss_row = results_df.loc[results_df["test_loss"].idxmin()]

display(
    Markdown(
        f'''
O MLP usa apenas camadas densas sobre os pixels achatados, portanto perde a organiz
A VGG16 e a ResNet18 usam transfer learning: as camadas convolucionais já aprendera
Comparando as regularizacoes da ResNet18, o Dropout reduz a dependência entre neuro
'''
    )
)
```

O MLP usa apenas camadas densas sobre os pixels achatados, portanto perde a organização espacial das imagens. Por isso, ele tende a apresentar desempenho inferior aos modelos com redes convolucionais pré-treinadas.

A VGG16 e a ResNet18 usam transfer learning: as camadas convolucionais já aprenderam filtros visuais gerais no ImageNet e funcionam como extratoras de características para o CIFAR-10. Na execução deste notebook, o melhor `test_acc` foi obtido por **VGG16**, com `test_acc = 0.8460`. O menor `test_loss` foi obtido por **ResNet18**, com `test_loss = 0.5583`.

Comparando as regularizações da ResNet18, o Dropout reduz a dependência entre neurônios do classificador, a L2 penaliza pesos muito grandes usando `weight_decay` no Adam, e a L1 favorece pesos mais esparsos ao adicionar uma penalização diretamente à perda. O impacto prático deve ser avaliado pela tabela: se uma técnica melhora `test_acc` ou reduz `test_loss` em relação à ResNet18 sem regularização, ela ajudou a controlar o sobreajuste; caso contrário, a regularização pode ter sido forte demais ou desnecessária para este classificador pequeno com a base convolucional congelada.